SP-108 Green ASM

Software Test Plan and Software Test Report

CS 4850 - Section 01 – Spring 2026

04/12/2026

Professor Perry

Alex Tidwell Team Leader Documentation + Developer (as needed)	Katherine Smith Documentation + Developer	Sumaya Tamenne Documentation	Phillip Eiser Developer	Brianna Lewis Developer

Table of Contents

1. Overview 2

1.1. Purpose 3

1.2. Scope 3

2. Testing Summary 3

2.1. Scope of Testing 4

3. Analysis of Scope and Test Focus Areas 4

3.1. Release Content 5

3.2. Regression Testing 5

3.3. Platform Testing 6

4. Progression Test Objectives 6

5. Other Testing 7

5.1. Stress & Volume Testing (S&V) 8

5.2. Connectivity Testing (CT) 8

5.3. Disaster Recovery/Back Up 8

5.4. Unit Testing 8

5.5. Integration Testing 8

6. Test Strategy 8

6.1. Test level responsibility 9

6.2. Test Type & Approach 9

6.3. Build strategy 9

6.4. Test Execution Schedule 10

6.5. Facility, data, and resource provision plan 10

6.6. Testing Tools 10

6.7. Testing Metrics 10

7. Test Environment Plan 10

7.1. Test Environment Diagram 11

7.2. Test Environment Details 11

7.3. Establishing Environment 12

7.4. Environment Control 12

7.5. Environment Roles and Responsibilities 12

8. Assumptions and Dependencies 13

8.1. Assumptions 14

8.2. Dependencies 14

9. Entry and Exit Criteria 14

9.1. Entry Criteria 15

9.2. Exit Criteria 15

10. Administrative Plan 15

10.1. Approvals 16

10.2. Test Milestones and Schedule 16

10.3. Training 16

10.4. Defect Management 17

11. Definitions 17

12. References 18

13. Points of Contact 19

14. Software Test Report (STR) 20

1. Overview

1.1. Purpose

The purpose of this document is to define the testing strategy for an Automated Security Monitor (ASM) system developed by our team. The testing will cover the ability of the system to scan the target environment and how successful the reporting system is end-to-end. The objective is to make sure that the ASM system can discover, enumerate, and report on the target environment that the team has made.
Each team member will be responsible for certain areas of execution and documentation for testing. There will be two types of testing: the team's weekly drift reports, which are automatically generated every Monday, and manual vulnerability scanning and verification. For testing to begin, the target environment will need to be running all Docker containers. The ASM machine needs to be able to reach the target VM over the network with all scanner material installed and running. Testing is complete when a summary report is sent to the Discord channel, indicating that OWASP ZAP has successfully run a scan and detected vulnerabilities. Manual testing can be performed by executing the bash script and verifying that the components of the ASM produce the correct output. At this point in the project, two websites are running on port 80 of the test environment. There is a known limitation that ZAP cannot detect both, as it can only test the first response it gets from that port. Our weekly drift reports will be the main way of verifying that the ASM system is working successfully, and with manual testing, the team will be able to confirm the scope of what the machine is capable of. From this extensive testing, the team will present some of the weekly drift reports, Ollama API summaries, as well as ZAPs scans, which are in JSON file format. There will also be confirmation that the ArcherySec is properly displaying the data on the deliverables from the OWASP ZAP scans.

1.2. Scope

This document details the testing that will be performed by the project team for the ASM project. It defines the overall testing requirements and provides an integrated view of the project test activities. The two main systems that will be tested throughout the process are the target environment itself, which includes WordPress, nginx, MySQL, Redis, and a static page. The next part to be tested would be the ASM pipeline, which includes OWASP ZAP scans, Ollama API report summary, Discord webhook, bash script, and ArcherySec. Another part that will be tested is Dnsmasq, which is responsible for DNS resolution. The team will do the testing through our weekly drift reports, which are scheduled for every Monday, as well as manual testing by running the bash script. Through this method, the team will be able to test how the data is presented and its accuracy. The resources needed are the target virtual machine with each Docker container running properly, and the ASM VM needs OWASP ZAP, Ollama, ArcherySec, and the bash script. The ASM VM and target VM both need to be on the same network, with dnsmasq doing DNS configuration on the target. There will also need to be a Discord to receive the report summaries. With this, all of these resources will be needed to properly go through with testing.

2. Testing Summary

2.1. Scope of Testing

2.1.1. In scope

There are a few areas that are currently in scope for testing. First, the target environment, it is in scope for us to scan and detect vulnerabilities from WordPress as well as the static page. Next, verifying the Ollama API correctly summarizes the ZAP scan results and outputs a readable report for us. Another is testing ArcherySec's ability to display and organize the data from our scans. Confirming that the full end-to-end pipeline works correctly is in scope. Testing the Discord webhook is working and is in scope. Lastly, dnsmasq is also in scope, as the project team can make sure that dnsmasq is correctly resolving the internal domain names.

2.1.2. Out of scope

Areas such as scanning any external environments or public servers are not within the scope of testing for this project. Security testing for the ASM system is also out of scope. Lastly, testing the detection of MySQL and Redis is out of scope, as these services have not yet been fully configured with necessary vulnerabilities. Though it is planned for the near future to implement these into the project.

3. Analysis of Scope and Test Focus Areas

3.1. Release Content

The release of this system is expected to include a fully functional Attack Surface Management System that continuously discovers assets within the target environment, analyses risk associated with assets, and alerts users of any changes. The system will integrate tools like Docker, OWASP ZAP, and Discord Webhooks to create the final working pipeline. Docker-based containers are used for the target environment, OWASP ZAP is used for scanning, Ollama API is used for summarization of data, and Discord Webhooks is used for alerting the user of any changes.

The release of this project also includes all documentation written during the length of the project. The final report includes the initial project plan, requirements and design documents, development document, and the test plan and report documents. With documentation included in the final release, reviewers and users can grasp the full intent behind each decision made throughout the implementation of the ASM.

A final tool that is expected to be added to our existing ASM is ArcherySec. ArcherySec can provide a more user-friendly dashboard to enhance the user's experience when analysing scan reports. The new addition supports a more real-world implementation of an ASM and allows for scalability.

3.2. Regression Testing

Regression testing is very important for this project since the ASM system is connected in a pipeline-like structure. Changes to one component of the system could drastically impact the entire workflow. The primary focus of the regression testing will be on OWASP ZAP and the raw scan reports provided. As explained in the development document, scans are initiated by the Cron job, making them the starting point of the system. Any changes made to scan configurations, or the target environment need to be tested to ensure it does not break the processing and alerting algorithms.

The primary regression testing areas include:

- Ensuring OWASP ZAP scans are successfully analysing the entire target environment
- Ensuring scan reports are created in the format that Ollama API can ingest
- Verifying the Python script can access the most recent raw scan file
- Checking Ollama API summaries for correctness and understanding
- Ensuring Discord Webhooks is alerting the user with the most recent scan update

Each of these areas should be tested after any modifications are made to the existing scripts. This includes any new tools added such as ArcherySec, any changes to scan configurations, or any additions to the target environment. This is important to ensure the current functionality remains intact as the system grows.

3.3. Platform Testing

Testing is always conducted in a controlled environment to avoid unexpected conditions impacting output. There are two virtual machines used, one that is running the ASM and the other that acts as the target environment. The target environment VM runs with Ubuntu Server 24.04 with Docker containers simulating vulnerabilities and services. The tools used during testing include OWASP ZAP for scanning, Docker Compose for target environment containers, Python scripts to transfer raw scanner data to Ollama API for summarization, and Discord Webhooks for alerting.

When testing we are ensuring all tools including OWASP ZAP, Docker Compose, Python, Ollama API, and Discord Webhooks are working properly and integrating with no errors. Using virtual machines allows for a controlled testing environment while still simulating a real-world ASM system. Additionally, separate VMs for the ASM and target environment allow testers to verify that scanning behaviour is accurate. Platform testing is done to ensure that all tools can work together and run smoothly from end-to-end. When performing platform testing, the goal is to verify compatibility of tools and proper communication between the ASM and the target environment.

4. Progression Test Objectives

Below are the main functions of the ASM that need to be tested along with their priority.

Ref	Function	Test Objective	Evaluation Criteria	X-Ref	P
1	OWASP ZAP Scanner	Make sure scanning of test environment works	OWASP ZAP scans the test environment with no errors and correct report is created.	Development Document 1.2	High
2	Python Script	Ensure Python script can access and process data	Python script can parse raw data from scanner report without errors	Test Plan 3.2	High
3	Ollama API Summarization	Confirm parsed scanner data is received a summarized	Summary of latest file is returned in simplified language for user to understand	Development Document 1.0.2	High
4	Webhooks Alerts	Ensure user is notified of weekly reports and any changes in the attack surface	Alert messages appear in the Discord channel with latest updates	Development Document 1.2.1	High
5	Target Environment Configuration	Ensure Docker-based target environment runs correctly	All containers are accessible when ASM is run	Development Document 1.1	High
6	Raw Report File	Ensure the most recent file is selected for processing	Latest file is selected with no errors regarding multiple files existing	Development Document 1.0	Medium
7	Summary Report File	Ensure summary reports are being saved	File with summary of raw data is saved with correct information	Development Document 1.0	Medium

5. Other Testing

5.1. Stress & Volume Testing (S&V)

In order to stress test our system we will need to add more containers with vulnerabilities to our system, as well as adding more vulnerabilities in the containers that are already there, so that we may see what limits our ASM has when it comes to detecting vulnerabilities on the network.

5.2. Connectivity Testing (CT)

To test our ASMs connection to the rest of the network and its external systems we will need to do various tasks. For testing network connectivity, we can ping the network or things on it with something like curl to check that we get a response from it. When testing API connections, we will have to manually validate that Ollama is getting our OWASP Zap files and seeing that the report generated properly outputs to Discord, verifying Discord webhooks.

5.3. Disaster Recovery/Back Up

We do not currently have a disaster recovery plan other than having potential backups of important files saved on other machines in case we need to use them.

5.4. Unit Testing

When adding new modules to our ASM we will ensure that given a specific form or file we need it to process, or network we need it to scan, can be done with the new module, as well as testing any outputs given with the next step.

5.5. Integration Testing

Any new modules added will once again be tested with the whole of the ASM in an automated manner after unit testing is done. Once we see that the data flow is going as expected, we will know the new addition was successfully implemented and integrated.

6. Test Strategy

6.1. Test level responsibility

Test Level	External Party	Proj Team	Business
Unit Testing		P	
Integration Testing		P	
Connectivity Testing		P	
Stress Testing		P	

6.2. Test Type & Approach

Test Type	Objectives
Unit Testing	When we desire to add a new module we will do the following, • Test that it takes the input we desire it to • Test it can output what we need it to for the next portion of the ASM • Verify that this setup can be automated
Integration Testing	Once a new module is integrated into the system, we will test that the system can run automated using scripts and that the data flows as expected and gives the expected outputs.
Connectivity Testing	We will test our various connections that include • The network to be scanned • Ollama connection • OWASP Zap connection • Discord Webhooks connection • ArcherySec • Any additionally needed connections
Stress Testing	Done to test the limits of the ASMs accuracy of finding vulnerabilities by adding both more vulnerabilities and containers.

6.3. Build strategy

The software will have to be built in multiple steps with certain verifications being done at each step.

1. Verify that you are using Ubuntu Server 24.04 LTS (Other versions may work but are untested)
2. Clone our repository from GitHub
3. Verify that you have Python 3.10 installed, if not install it
4. Make sure the port 53 is open and unused
5. Make sure all dependencies are setup properly and any Api keys needed are setup as environment variables
6. Set up docker compose files to scan (or just scan network with permission)
7. Run docker compose files
8. Set up cron job to execute bash scripts

9. Run scan to test everything is set up properly

6.4. Test Execution Schedule

Test scans of the system are run on every Monday; however, scans can be initiated as needed if we add new modules or functions to the system.

6.5. Facility, data, and resource provision plan

6.5.1. Test environment

Our testing environment runs by using a VPN to gain remote access to a VM that our ASM is on. We run our tests with Discord and ArcherySec open to verify that the proper outputs are received.

6.5.2. Testing Requirements

- VPN into the virtual machine through Command Line
- Run test
- Compare found CVEs to vulnerabilities we know are on network
- Check that information properly uploads to ArcherySec

6.5.3. Resources & Skills

- Knowledge of CLI commands needed

6.6. Testing Tools

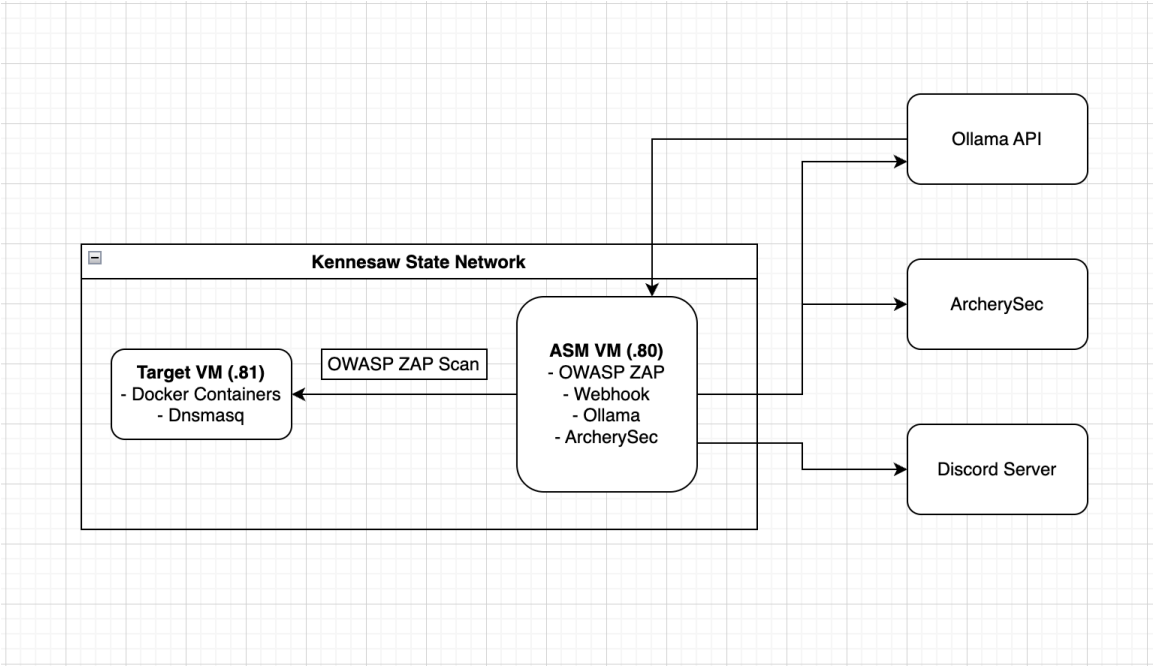
Process	Tool
Test running	Manually or automatically on a virtual machine
Test result tracking	Discord and ArcherySec
Defect management	Microsoft Word

6.7. Testing Metrics

Our metric for testing will be CVEs found / Vulnerabilities known to be on network, with an acceptable score of around 80% to pass currently.

7. Test Environment Plan

7.1. Test Environment Diagram



7.2. Test Environment Details

7.2.1. Testers

There are three testers involved in testing. Each tester must SSH into both VMs, including the target VM and the ASM system VM. The testers must also be able to access the Discord channel to review the reports. For hardware requirements testers will need a personal computer that can SSH into the VMs, which can be done from most modern computers as long as they have a VPN connected to Kennesaw State Universities IP address.

7.2.2. Hardware and Firmware

There are two virtual machines hosted on the schools' network. A VPN is needed to connect to the school's internet to access both VMs. The ASM VM runs the scanning tools and oversees the ASM pipeline. While the target VM hosts the Docker containers with the target environment. Both VMs are accessible via SSH from a computer with a CLI. The VMs are currently running Ubuntu 22.04.

7.2.3. Software

Software Name	Version	Item
Ubuntu	22.04	Operating System
Docker Compose	v5.1.2	Container manager
WordPress	6.9.1	Content Management System
MySQL	8.0	Target Database
Nginx	1.29.6	Web Server

Redis	7.4	Caching system
OWASP ZAP	2.17.0	Vulnerability Scanner
Ollama	0.20.6	AI for report summarization
ArcherySec	2.0.6	Vulnerability Management Display
Dnsmasq	2.90	DNS resolution

7.2.4. Interfaces

ArcherySec is a vulnerability management display which is being used by the team for the ASM system. It receives data from the OWASP ZAP scans and displays the data alongside the various vulnerability findings. The team will be the one to establish this interface and the ZAP scans will be integrated prior to testing.

7.3. Establishing Environment

Task	Requirements	Responsibility	Start Date	End Date
Deploy target VM	Docker, Docker-compose	Target environment developer	02/01/26	In progress
Configure dnsmasq	Ubuntu, dnsmasq	Target environment developer	02/01/26	In progress
Configure OWASP ZAP	ASM VM, OWASP ZAP installed	ASM developer	02/01/26	02/22/26
Configure Ollama API	ASM VM, Ollama installed	ASM developer	02/01/26	02/22/26
Configure Discord webhook	ASM VM, Discord Server	ASM developer	02/01/26	02/22/26
Configure ArcherySec	ASM VM, ArcherySec installed, ArcherySec account	ASM developer	04/01/26	In progress
Cron Job	ASM VM, Cron installed	ASM developer	02/01/26	02/22/26

7.4. Environment Control

Software versions are controlled via the Docker compose file in the target environment. Software versions for the ASM system are kept on the same version to avoid any unwanted complications from switching versions during testing phase. Environments are strictly accessed by team members through SSH with the IP addresses of each machine. The target environment is monitored every week through the weekly drift reports produced by the ASM machine.

7.5. Environment Roles and Responsibilities

Define the roles and responsibilities of persons who will be responsible for, or interface with the environment

Role	Staff Member	Responsibilities
Target Environment Manager	Katherine Smith	Responsible for overall establishment, coordination and support of the target environment through maintaining the Docker containers and dnsmasq.
ASM Manager	Brianna Lewis	Responsible for over ASM environment issues, testing functionality, research, and programming for environment.
ASM Manager	Philip Eiser	Responsible for OWASP ZAP, Ollama, bash script, and report delivery.

8. Assumptions and Dependencies

8.1. Assumptions

There were many assumptions made when creating this project that still hold when testing.

First, we assume that all testers are able to troubleshoot when errors or configuration issues occur during testing. We also assume that the testing environment is correctly set up. Proper set up includes proper installation of Docker, Python, OWASP ZAP, and Ollama with all requirements mention in the development document. Another assumption is that external APIs and services including Ollama API and Discord Webhooks. Other assumptions include that the cron job is properly working, OWASP ZAP is successfully generating scan reports and users have permission run the system when they choose. Lastly, the system assumes the target environment is running with services and vulnerabilities ready for scanning. While the ASM is intended to help catch possible attacks, it still requires a user knowledgeable in cybersecurity incident response to make use of the reports given by the ASM. This system is just a tool and is not intended to prevent threats on its own.

8.2. Dependencies

The ASM depends on many tools and configurations to run properly when testing. First, testers depend on access to both the ASM VM and target environment VM. Tools such as OWASP ZAP and Ollama API are dependencies of testing including their installations and configurations. The ASM also requires a valid Discord Webhooks URL to alert the user. All Python scripts, bash scripts, and cron jobs must be properly configured in order for testing to be conducted. For Python, version 3.10 is required along with specific libraries such as requests, Json, os, and glob libraries. Other dependencies include access to the internet for communication with APIs, connection to KSU VPN to gain access to the virtual machines, and a sufficient device to run Docker containers.

Overall testing depends on the system tools integrating successfully. As stated, the components of this system rely on the previous step to run with no errors to continue. Failure of one component will impact the entire system, making integration a large dependency for testing.

9. Entry and Exit Criteria

9.1. Entry Criteria

Testing will begin once a few criteria are met. To start off the target VM must have all Docker containers up and running successfully with WordPress, nginx, MySQL, Redis, and the static page. Next the ASM VM needs to have direct access via the network to the target VM and DNS resolution needs to be working correctly from dnsmasq. OWASP ZAP, Ollama, and ArcherySec need to be installed on the ASM machine and configured to properly execute scans and send reports for the project team. Next the bash script needs to be confirmed working correctly and able to execute commands for the scans to run. The Discord webhook also needs to be configured and able to reach the Discord server. Lastly, the team needs to be able to access both VMs via SSH and need to have a connection to Kennesaw State University IP address, either through VPN or by being onsite.

9.2. Exit Criteria

Testing will be considered complete once a few conditions are met. First OWASP ZAP has successfully run a full scan on the target environment and has produced a JSON report. Next the Ollama API has been sent the JSON report and generates a readable summary from that ZAP scan. This summary should be sent to the Discord server via the webhook, once this is complete this would mean success for the area. ArcherySec should correctly display vulnerability data from the ZAP scan. Weekly drift reports also need to be correctly sent to the discord channel every Monday. Lastly, it is essential that all configured vulnerabilities from the target environment are detected by the ZAP scan.

10. Administrative Plan

10.1. Approvals

Detail the responsibilities for testing signoff. For example, the following persons are responsible for the critical aspects of testing:

Task	Responsible Person	Escalation/ Approver
Systems Integration Signoff	Development Team	Team Leader
User Acceptance Testing Signoff	Documentation Team	Team Leader
Production Verification Testing Signoff	Development Team	Team Leader
OWASP ZAP Scanner Testing Signoff	Development Team	Team Leader
Python Processing Script Testing Signoff	Development Team	Team Leader
Ollama API Integration Testing Signoff	Development Team	Team Leader
Webhooks Alerting Testing Signoff	Documentation Team	Team Leader
Target Environment Setup Testing Signoff	Documentation Team	Team Leader
Raw and Summary File Testing Signoff	Development Team	Team Leader

10.2. Test Milestones and Schedule

Detailed below are the high-level testing milestones.

Milestone	Planned End Date	Actual End Date	Resource
Target Environment Setup	2/8/2026	2/15/2026	Dev Team
OWASP Scan Testing	3/1/2026	3/1/2026	Dev Team
Python Script Testing	3/1/2026	3/1/2026	Dev Team
Ollama API Testing	3/8/2026	3/8/2026	Dev Team
Webhooks Alerting Testing	3/15/2026	3/31/2026	Entire Team
Final ASM Testing	4/5/2026	3/31/2026	Entire Team

10.3. Training

The following training requirements have been identified to ensure testing can commence:

Training Requirement	Staff	Date
Docker Container Setup and Configuration	Development Team	2/8/2026
OWASP ZAP Call and Usage	Development Team	3/1/2026
Python Script Execution	Development Team	3/1/2026
Ollama API Integration	Development Team	3/8/2026

Training Requirement	Staff	Date
Discord Webhooks Setup & Testing	Entire Team	3/15/2026

10.4. Defect Management

If a defect is discovered during testing by any team member, it will be documented. Whether through our Discord chat, a separate document, or in our meeting notes, the issue will be discussed among the team. Large defects like issues with Python scripts, OWASP ZAP, Ollama API, or the target environment setup, will take priority because they impact overall functionality of the ASM. Issues in any of these areas will cause the system to crash. Other issues such as improper formatting of alerts in Discord Webhooks are lower-priority and will be handled last. After a defect has been solved, the system will be tested again, from end-to-end, to ensure all failures have been addressed and no new issues were introduced in the process. This system of defect management allows for all team members to be aware of defects and understand what issues take priority over others.

11. Definitions

The following acronyms and terms have been used throughout this document

Term/Acronym	Definition
ASM	Attack Surface Manager
CVE	Common Vulnerabilities and Exposures
VM	Virtual Machine
API	Application Programming Interface
CLI	Command Line Interface
VPN	Virtual Private Network
CMS	Content Management System

12. References

The following documents have been used to assist in creation of this document.

#	Document name	Version	Comments
1	SP-108-Green-ASM-Development-Document	Final	Used for reference of past work
2	Gantt Chart	4	Used for project and testing plan related sections

13. Points of Contact

The following people can be contacted in reference to this document

Primary Contact	
Name	Alex Tidwell
Title/Organisation	Team Leader/Documentation
Phone	678-773-1588
Email	Mtidwe12@students.kennesaw.edu
Secondary Contact	
Name	
Title/Organisation	
Phone	
Email	

14. Software Test Report (STR)

Requirement	Pass	Fail	Severity
Login to VM on VPN	Yes	No	N/A
Initiate Scan	Yes	No	N/A
OWASP Zap scan outputs	Yes	No	N/A
Ollama processes scan to create report	Yes	No	N/A
Report is outputted to discord	Yes	No	N/A
ArcherySec receives report and scan information	Yes	No	N/A
Scan runs autonomously as well	Yes	No	N/A